

Project: Learning Steps Origins

A cybersteps project

Date: 26. February 2026

Participants: Gregor

Executive Summary

Today, I deployed the **LearningSteps FastAPI application** to a secure **2-tier architecture** in Microsoft Azure. The focus was on:

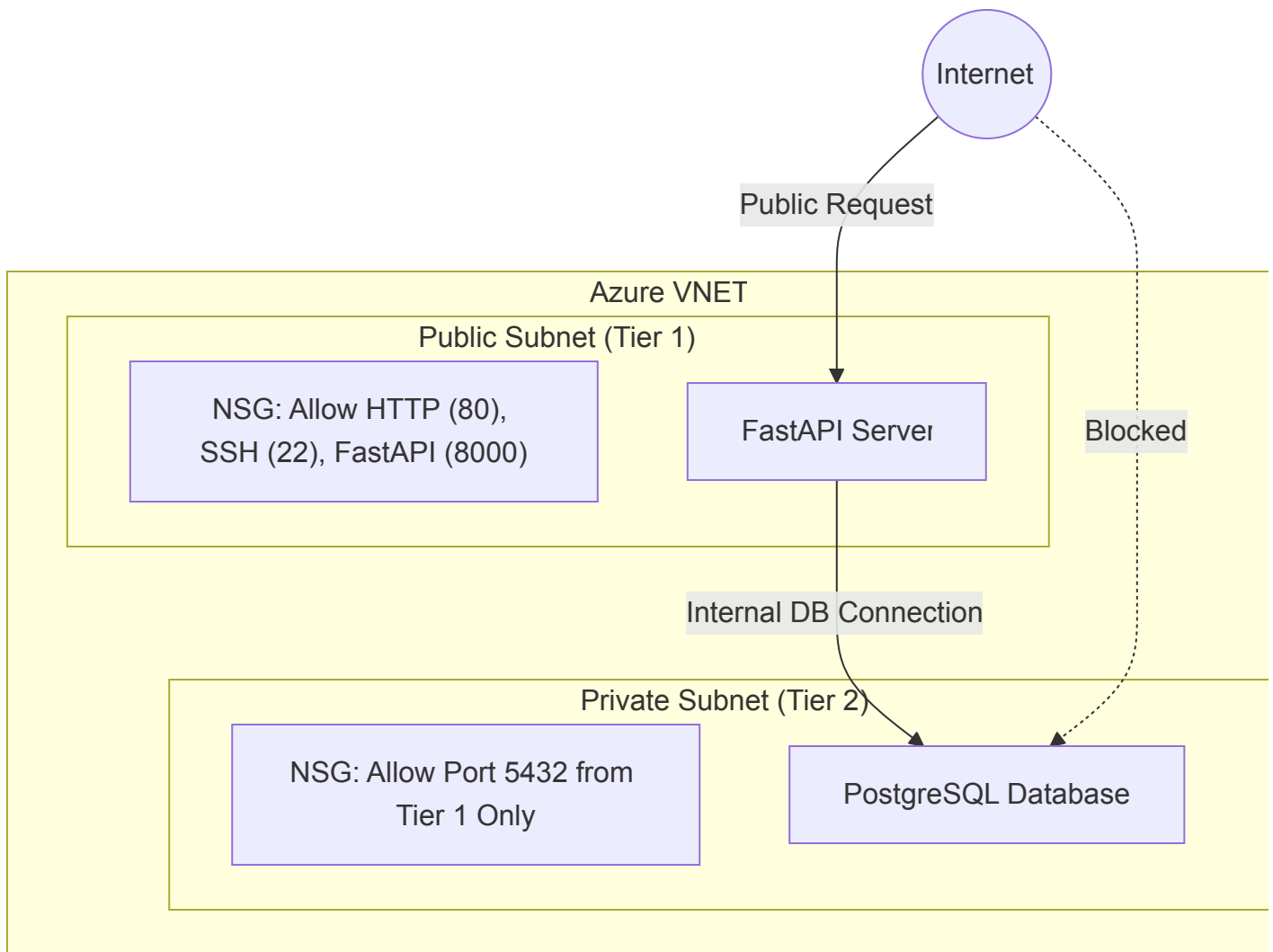
Network segmentation (public vs. private subnets)

Secure connectivity between the application server and database

Persistent storage for journal entries

Running the app in a **production-ready manner** on a VM

This forms the foundation for future enhancements such as TLS, reverse proxy, CI/CD, and observability.



- **Tier 1:** appVM running FastAPI with public IP (port 8000)
- **Tier 2:** dbVM running PostgreSQL, private subnet only
- **NSGs** enforce least-privilege access

Infrastructure

1. Created Azure **Resource Group**: project_LearningStepsRG
2. Created **Virtual Network** with:
 - **Public Subnet** → appVM
 - **Private Subnet** → dbVM
3. Configured **Network Security Groups (NSGs)**:
 - appVM NSG allows SSH (22), HTTP (80), and FastAPI (8000)
 - dbVM NSG allows PostgreSQL (5432) only from appVM subnet

Application Setup (Tier 1)

1. Deployed **FastAPI** on appVM without Docker (direct Python runtime)

2. Installed dependencies in a virtual environment (venv)
3. Configured .env with private DB connection string:

```
DATABASE_URL=postgresql://journaluser:StrongPassword123!@10.0.2.4:5432/journalingdb
```

4. Tested **database connectivity** from appVM:

```
psql -h 10.0.2.4 -U journaluser -d journalingdb
```

5. Configured **systemd service** to run FastAPI on boot:

```
sudo systemctl enable learningsteps  
sudo systemctl start learningsteps
```

6. Opened **port 8000** in NSG for public access

Database Setup (Tier 2)

1. Deployed **PostgreSQL** on a VM without public IP
2. Configured pg_hba.conf and postgresql.conf to:
 - Accept connections from the private appVM subnet
 - Require password authentication
3. Created **journal database and user**:

```
CREATE DATABASE journalingdb;  
CREATE USER journaluser WITH ENCRYPTED PASSWORD '#####';  
GRANT ALL PRIVILEGES ON DATABASE journalingdb TO journaluser;
```

4. Initialized **entries table** and indexes manually from SQL script:

```
CREATE TABLE IF NOT EXISTS entries (  
    id VARCHAR PRIMARY KEY,  
    data JSONB NOT NULL,  
    created_at TIMESTAMP WITH TIME ZONE NOT NULL,  
    updated_at TIMESTAMP WITH TIME ZONE NOT NULL  
);  
CREATE INDEX IF NOT EXISTS idx_entries_created_at ON entries(created_at);  
CREATE INDEX IF NOT EXISTS idx_entries_data_gin ON entries USING GIN (data);
```

Validation

- API is accessible via **appVM public IP**
- CRUD operations on journal entries work correctly
- PostgreSQL is **not publicly accessible**
- Data persists across VM reboots
- Network segmentation between Tier 1 and Tier 2 is enforced

Next Steps / Future Improvements

- Add **TLS/HTTPS** for API requests
- Optionally add **NGINX reverse proxy** on port 80/443
- Add **monitoring and logging** for observability
- Add a Application Gateway
- Add a bastion server

Outcome: A secure, production-ready 2-tier IaaS architecture on Azure, with FastAPI connected to a private PostgreSQL VM — perfectly aligned with course objectives on **networking, security, and multi-tier architecture**.